

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA0307	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	Shaik Nuha	Reg. No.:	192472242
Section / Batch:	B.E-CSE-AI	Date:	

Code of Conduct

I Shaik Nuha(192472242) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sk.Nuha

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
 - GitHub Repository Link in GCR
 - Google Colab Share Link in GCR
 - Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1
Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject–verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2
Probabilistic Context-Free Grammars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction

Experiment Description

Design a Python program to validate whether a sentence follows the given Context-Free Grammar (CFG) rules and generate its parse tree representation.

Grammar:

```
S → NP VP
NP → Det N
VP → V NP

Det → the | a
N → student | teacher | book
V → reads | likes
```

Task

1. Accept a sentence as input.
2. Verify whether it conforms to the CFG.
3. Construct and return the parse tree.
4. Return "Invalid Sentence" if parsing fails.

Function Prototype

```
def parse_cfg(sentence):
    pass
```

Test Cases

Input	Expected Output
the student reads a book	Valid Parse Tree
a teacher likes the book	Valid Parse Tree
student reads book	Invalid Sentence
the book likes a teacher	Valid Parse Tree
reads the student book	Invalid Sentence

Experiment 2: Agreement and Feature Structure Checking

Experiment Description

Design a Python program that verifies subject–verb agreement using feature structures containing Number and Person attributes.

Feature Rules

```
he → singular, third person
she → singular, third person
it → singular, third person
they → plural

runs → singular verb
writes → singular verb

run → plural verb
write → plural verb
```

Task

1. Accept Subject and Verb.

2. Compare feature structures.
3. Return True if agreement holds.
4. Return False otherwise.

Function Prototype

```
def check_agreement(subject, verb):
    pass
```

Test Cases

Subject Verb Expected

he	runs	True
he	run	False
they	run	True
they	writes	False
she	writes	True

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

Experiment Description

Implement a Python program to compute the probability of a sentence using the given PCFG.

Grammar

S	→ NP VP	[1.0]
NP	→ John	[0.6]
NP	→ Mary	[0.4]
VP	→ runs	[0.5]
VP	→ walks	[0.5]

Probability Formula

$$\begin{aligned}
 &P(\text{Sentence}) \\
 &= \\
 &P(S \rightarrow NP \ VP) \\
 &\times P(NP) \\
 &\times P(VP)
 \end{aligned}$$

Task

1. Accept a two-word sentence.
2. Parse using the PCFG.
3. Compute sentence probability.
4. Return 0 if sentence cannot be generated.

Function Prototype

```
def sentence_probability(sentence):
    pass
```

Test Cases

Input Sentence Expected Probability

John runs	0.30
John walks	0.30
Mary runs	0.20
Mary walks	0.20
Peter runs	0.00

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1.Context-Free Grammar (CFG) Rule Validation and Parse Tree Construction

Aim: To design a Python program that validates whether a given sentence follows the specified Context-Free Grammar (CFG) rules and generates its parse tree representation.

Theory:

A Context-Free Grammar (CFG) is a formal grammar used in Natural Language Processing (NLP) to describe the syntactic structure of sentences. A CFG consists of production rules that define how symbols can be replaced to generate valid sentences. Parse trees visually represent the grammatical structure of a sentence according to the grammar rules.

Grammar Used:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$VP \rightarrow V NP$

$Det \rightarrow the \mid a$

$N \rightarrow student \mid teacher \mid book$

$V \rightarrow reads \mid likes$

Algorithm:

1. Read the input sentence.
2. Split the sentence into words.
3. Check whether the sentence contains exactly five words.
4. Verify each word according to the CFG rules.
5. If all rules are satisfied, construct and display the parse tree.
6. Otherwise, display "Invalid Sentence".

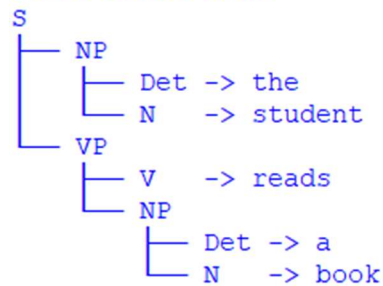
Python Program:

```
def parse_cfg(sentence):
    words = sentence.lower().split()
    determiners = ["the", "a"]
    nouns = ["student", "teacher", "book"]
    verbs = ["reads", "likes"]
    if len(words) != 5:
        return "Invalid Sentence"
    det1, noun1, verb, det2, noun2 = words
    if (det1 in determiners and
        noun1 in nouns and
        verb in verbs and
        det2 in determiners and
        noun2 in nouns):
        print("\nValid Parse Tree")
        print("S")
        print("├── NP")
        print(f"│   ├── Det -> {det1}")
        print(f"│   └── N   -> {noun1}")
        print("└── VP")
        print(f"    ├── V   -> {verb}")
        print(f"    └── NP")
        print(f"        ├── Det -> {det2}")
        print(f"        └── N   -> {noun2}")
    else:
        print("Invalid Sentence")
sentence = input("Enter a sentence: ")
parse_cfg(sentence)
```

Sample Output

Enter a sentence: the student reads a book

Valid Parse Tree



Result

The program successfully validates whether a sentence follows the given CFG and generates the corresponding parse tree. Invalid sentences are correctly rejected.

2.Agreement and Feature Structure Checking

Aim: To develop a Python program that verifies subject–verb agreement using feature structures containing Number and Person attributes.

Theory

Feature structures represent grammatical properties such as number (singular/plural) and person (first, second, third). Subject–verb agreement ensures that the verb matches the subject in these grammatical features. This process is essential in Natural Language Processing for grammar checking and sentence validation.

Feature Rules:

Subject	Features
he	Singular, Third Person
she	Singular, Third Person
it	Singular, Third Person
they	Plural

Verb	Type
runs	Singular
Writes	Singular
Run	Plural
Write	Plural

Algorithm:

1. Read the subject and verb.
2. Store the grammatical features of subjects and verbs.
3. Compare the number feature of the subject with the verb.
4. If both match, return True.
5. Otherwise, return False.

Python Program:

```
def check_agreement(subject, verb):
    subject_features = {
        "he": ("singular", "third"),
        "she": ("singular", "third"),
        "it": ("singular", "third"),
        "they": ("plural", "third")
    }
    verb_features = {
        "runs": "singular",
        "writes": "singular",
        "run": "plural",
        "write": "plural"
    }
    subject = subject.lower()
    verb = verb.lower()
    if subject not in subject_features:
        return False
    if verb not in verb_features:
        return False
    return subject_features[subject][0] == verb_features[verb]
subject = input("Enter Subject: ")
verb = input("Enter Verb: ")
print(check_agreement(subject, verb))
```

Sample Output

```
Enter Subject: she
Enter Verb: writes
True
```

Result:

The program successfully verifies subject–verb agreement using feature structures and correctly identifies valid and invalid grammatical combinations.

3.Probabilistic Context-Free Grammar (PCFG) Parsing

Aim: To implement a Python program that computes the probability of a sentence using a Probabilistic Context-Free Grammar (PCFG).

Theory:

A Probabilistic Context-Free Grammar (PCFG) extends a CFG by assigning probabilities to each production rule. The probability of a sentence is obtained by multiplying the probabilities of the grammar rules used in its derivation. PCFGs are widely used in NLP applications such as speech recognition, machine translation, and syntactic parsing.

Grammar:

$S \rightarrow NP VP$ [1.0]

$NP \rightarrow John$ [0.6]

$NP \rightarrow Mary$ [0.4]

$VP \rightarrow runs$ [0.5]

$VP \rightarrow walks$ [0.5]

Probability Formula:

$P(\text{Sentence}) = P(S \rightarrow NP VP) \times P(NP) \times P(VP)$

Algorithm:

1. Read the input sentence.
2. Split the sentence into subject and verb.
3. Verify whether both words exist in the grammar.
4. Compute the sentence probability using the PCFG formula.
5. Display the calculated probability.
6. If the sentence cannot be generated, return 0.00.

Python Program:

```
def sentence_probability(sentence):
    grammar = {
        "John": 0.6,
        "Mary": 0.4
    }
    verbs = {
        "runs": 0.5,
        "walks": 0.5
    }
    words = sentence.split()
    if len(words) != 2:
        return 0.0
    subject, verb = words
    if subject in grammar and verb in verbs:
        probability = 1.0 * grammar[subject] * verbs[verb]
        return probability
    return 0.0
sentence = input("Enter Sentence: ")
print("Probability =", sentence_probability(sentence))
```

Sample Output


```
Enter Sentence: Mary runs  
Probability = 0.2
```

Result

The program successfully computes the probability of valid sentences using the given PCFG and returns 0.00 for sentences that are not generated by the grammar.